



INDEX TABLE

S.No.	Topic Name
1.	Behind Story of UI policy
2.	Introduction to UI Policies
3.	UI Policies: Fields and Controls
4.	Introduction to Data Policies
5.	Execution order of UI Policies
6.	Conversion between Data Policies and UI Policies
7.	Advanced Topics: Metadata, Migration and Debugging

1. Behind Story of UI Policy: Why Do We Need Policies?

Before diving into configurations, it is critical to understand the architectural gap these tools fill. ServiceNow administrators manage data integrity and user experience across two distinct environments: the Client (Browser) and the Server (Database).

The Business Scenario: Incident Management

Imagine you are building an Incident Management process for a large enterprise. You have two strict requirements:

1. **The UX Requirement:** When a Help Desk agent selects "Hardware" as the Category on a form, the "Configuration Item" field must appear and become mandatory. If they change it to "Software," the field should hide.
2. **The Integrity Requirement:** Every Incident, whether created manually by an agent, via an email integration, or through a REST API from a monitoring tool (like SolarWinds), must have a "Short Description."

The Challenge Without Policies

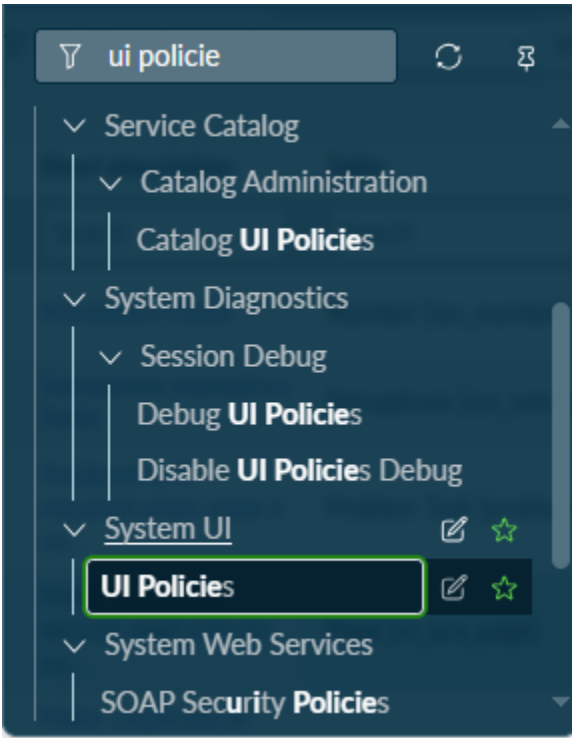
- **Solving the UX (Client):** You could write a JavaScript Client Script. However, scripts are harder to maintain, prone to syntax errors, and can slow down page load times if not optimized.
- **Solving the Integrity (Server):** You could write a Business Rule to abort the database transaction if the description is missing. However, this requires scripting knowledge and doesn't inherently give the user a clean visual cue on the form before they hit "Submit."

The Solution

- UI Policies handle the UX Requirement (Client-side, no-code, dynamic visibility).
- Data Policies handle the Integrity Requirement (Server-side, no-code, universal enforcement).

2. Introduction and Core Mechanisms

2.1. UI Policies (Client-Side Control)



Attribute	Details
1. Backend Table Name	sys_ui_policy
2. Action Table	sys_ui_policy_action
3. Navigation Path	System UI → UI Policies
4. Function	Provide non-scripted client-side logic to dynamically manage field

	attributes (Mandatory, Read-only, Visibility) on forms and lists.
--	---

2.2. Data Policies (Server-Side Control)

Attribute	Details
1. Clear Title	Data Policies: Server-Side Data Integrity Enforcement
2. Backend Table Name	sys_data_policy2
3. Rule Table	sys_data_policy_rule
4. Navigation Path	System Policy → Rules → Data Policies
5. Function	Provide non-scripted server-side logic to strictly enforce data integrity (Mandatory, Read-only) across all data input channels. Cannot control Visibility.
6. Real-World Usage	Ensuring data consistency when records are created via Forms, REST APIs, or Import Sets.

3. Deep Dive: UI Policy Form Fields

A UI Policy is defined on the sys_ui_policy table. The fields are split between the main definition and the execution settings.

3.1. Basic Definition & Identification Fields

The screenshot shows the ServiceNow 'UI Policy - New Record' form. The form is divided into two main sections: 'When to Apply' and 'Script'. The 'When to Apply' section contains fields for 'Table' (labeled 2), 'Application' (labeled 6), 'Active' (labeled 3), 'Short description' (labeled 1), and 'Order' (labeled 4). The 'Script' section contains a 'Conditions' field (labeled 5) with a dropdown menu and buttons for 'Add Filter Condition' and 'Add OR Clause'. Below the conditions are several checkboxes: 'Global' (checked), 'Reverse if false' (checked), 'On load' (checked), and 'Inherit' (unchecked). A 'Submit' button is at the bottom left. The top navigation bar shows 'servicenow' and 'UI Policy - New Record'.

Field Name	Type	Explanation	Backend Field
1. Short description	String	A clear, descriptive name for the policy's function. Best Practice: Use a structured naming convention (e.g., INC - CategoryHardware - Make CIMandatory).	short_description
2. Table	Reference	The target table where this policy will run (e.g., incident). Policies run on the selected table and its child tables (due to inheritance).	table

3. Active	Boolean	Controls whether the UI Policy is currently enabled. If unchecked, the policy is ignored during execution.	active
4. Order	Integer	Defines the sequence of execution. Policies with a lower order number run before those with a higher number. Crucial for resolving conflicting actions.	order
5. Condition	Condition Builder	Defines the criteria that must be met for the policy actions to apply (e.g., State is New). If empty, the policy always runs.	condition
6. Application	Reference	The scope this policy belongs to. Default is Global unless created within a custom application scope.	sys_scope

3.2. Execution and Control Fields

These checkboxes control *when* and *where* the policy runs.

UI Policy
asdjklthkashdjkf View: Advanced

Order 100

4

When to Apply Script

Conditions Add Filter Condition Add OR Clause

If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific

1 Global ☒

Reverse the effects of the UI policy actions when the Conditions evaluate to false

3 Reverse if false ☒

5 On load ☒

Tables that extend the specified Table inherit this UI Policy

6 Inherit ☐

Update Delete

Related Links

[Convert this to Data Policy](#)

[Default view](#)

[Run Point Scan](#)

[\(SN Links\) Versions \(1\)](#)

UI Policy Actions UI Policy Related List Actions

for text Search

UI policy = asdjklthkashdjkf

Field Name	Type	Explanation	Backend Field
1. Global	Boolean	If checked, the policy applies to all views (default behavior). If unchecked, the View field appears, allowing restriction to a specific form view (e.g., Self Service view only).	global
2. View	Reference	Appears only if Global is unchecked. Allows the policy to be restricted to a named view.	view
3. Reverse if false	Boolean	Highly recommended. If checked, the system automatically reverses the actions (mandatory → optional, read-only → editable, hidden → visible) when the	reverse_if_false

		Condition is no longer met (false).	
4. Run scripts	Boolean	Not covered in detail as per prompt, but included for completeness. Enables the Script tab, allowing advanced JavaScript logic to run before and after policy actions.	run_scripts
5. Run on load	Boolean	If checked (default), the policy executes immediately when the form loads, setting the initial state of fields.	onload
6. Inherit	Boolean	If checked (default), the policy is inherited and applied to all child tables of the specified Table .	inherit

3.3. UI Policy Actions (Related List)

Located on the sys_ui_policy_action table, defining *what* the policy does.

UI Policy
asdjklhaskdjklf

View: Advanced

Order100

When to Apply

Script

Conditions

Add Filter Condition

Add OR Clause

If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific

Global☒

Reverse the effects of the UI policy actions when the Conditions evaluate to false

Reverse if false☒

Apply the UI policy actions when the form is loaded and when the user changes values on the form

On load☒

Tables that extend the specified Table inherit this UI Policy

Inherit☐

Update

Delete

Related Links

[Convert this to Data Policy](#)

[Default view](#)

[Point Scan](#)

[\[Show\] Versions \(1\)](#)

UI Policy Actions

UI Policy Related List Actions

for text

Search

UI policy = asdjklhaskdjklf

servicenow

All

Favorites

History

Workspaces

UI Policy Action - New Record

Application scope: Technova Pr...

Update set: Default 1 [Technov...

Search

Submit

UI Policy Action

New record

Submit

UI policy

sfasfasf

Table

* Field name

Click to select...

1

5 Application

Technova Project Management

2 Mandatory

Leave alone

4 Visible

Leave alone

3 Read only

Leave alone

6 Clear the field value☐

Field Name	Action	Explanation

1. Field name	Reference	The specific field on the form the action targets.
2. Mandatory	Choice (True/False/Leave alone)	Controls the field's required status. True enforces it; False removes it.
3. Read only	Choice (True/False/Leave alone)	Controls the field's editability. True prevents user input.
4. Visible	Choice (True/False/Leave alone)	Controls whether the field is displayed on the form. False hides it.
5. Application	Global (By Default)	Defines the scope of the ui policy action.
6. Clear the field value	Checked by default	This empties the value of a specified field when the UI policy's conditions are met. This is useful for resetting a field when a user makes a choice that makes a previous value irrelevant.

4. Deep Dive: Data Policy Form Fields

A Data Policy is defined on the sys_data_policy2 table. It enforces server-side rules.

4.1. Basic Definition & Identification Fields

dev210379.service-now.com/now/nav/ui/classic/params/target/sys_data_policy2.do%3Fsys_id%3D-1%26sys_is_list%3Dtrue%26sys_target%3Dsys_...

servicenow All Favorites History Workspaces Data Policy - New Record Application s... Update set... Search

Data Policy New record

2 * Table -- None --

Inherit ☐

Reverse if false ☒

Active ☒ **3**

1 Short description

Description 

4 Conditions [Add Filter Condition](#) [Add OR Clause](#)

5 Application Technova Project Management

Apply to import sets ☒

Apply to SOAP ☒

Use as UI Policy on client ☒

[Submit](#)

No templates are available [Create A New One?](#)

Field Name	Type	Explanation	Backend Field
1. Short description	String	Descriptive name for the policy's purpose.	short_description
2. Table	Reference	The target table (e.g., cmdb_ci).	table
3. Active	Boolean	Controls whether the Data Policy is currently enforced.	active
4. Condition	Condition Builder	Criteria that must be met for the policy rules to apply.	condition
5. Application	Reference	The scope this policy belongs to.	sys_scope

4.2. Enforcement Control Fields

These control *how* and *where* the enforcement happens.

The screenshot shows the 'Data Policy - New Record' form in ServiceNow. The form is titled 'Data Policy - New Record' and has a 'Submit' button at the bottom left. The form includes several fields and checkboxes:

- Table:** A dropdown menu with '-- None --' selected.
- Inherit:** A checkbox labeled '1' with the value 'Inherit'.
- Reverse if false:** A checkbox labeled '2' with the value 'Reverse if false'.
- Active:** A checkbox labeled '3' with the value 'Active'.
- Short description:** A text input field.
- Description:** A text input field with a code icon.
- Conditions:** Two buttons: 'Add Filter Condition' and 'Add OR Clause'.
- Application:** A dropdown menu with 'Technova Project Management' selected.
- Apply to import sets:** A checkbox labeled '4' with the value 'Apply to import sets'.
- Apply to SOAP:** A checkbox labeled '5' with the value 'Apply to SOAP'.
- Use as UI Policy on client:** A checkbox labeled '3' with the value 'Use as UI Policy on client'.

Field Name	Type	Explanation	Backend Field
1. Inherit	Boolean	If checked (default), the policy is inherited by all child tables.	inherit
2. Reverse if false	Boolean	If checked, the actions are reversed if the condition is not met. Only applies when Use as UI Policy is also checked.	reverse_if_false
3. Use as UI Policy	Boolean	If checked, the Data Policy's rules are also enforced on the client-side form, mimicking a	use_as_ui_policy

		UI Policy's mandatory and read-only behavior. This is the internal mechanism used for Convert to UI Policy.	
4. Apply to import sets	Server-side Business Rules	If checked, the Business Rule executes when data is inserted or updated via an Import Set operation (e.g., loading data from an Excel spreadsheet).	
5. Apply to SOAP	Server-side Business Rules	If checked, the Business Rule executes when data is manipulated (Insert, Update, Delete) via an inbound SOAP Web Service request	

4.3. Data Policy Rules (Related List)

Located on the data_policy_rule table, defining *what* is enforced.

servicenow All Favorites History Workspaces Admin Data Policy

Search

Data Policy

Description

Conditions

Add Filter Condition Add OR Clause

-- choose field -- -- oper -- -- value --

Update Delete

Related Links

Convert this to UI Policy (SN Utils) Versions

Data Policy Rules for text Search

Data Policy = (empty)

Field name	Mandatory	Read only
No records to display		

No templates are available Create A New One?

servicenow All Favorites History Workspaces Data Policy Rule - New Record

Application scope: Global Update with Default (Global)

Data Policy Rule New record

Table Payoneer [x_1862731_techno_0_payoneer]

1 * Field name Click to select...

4 Application Global

23 Read only Leave alone

Mandatory Leave alone

Submit

Field Name	Action	Explanation
1. Field name	Reference	The specific field on the table the rule targets.
2. Mandatory	Choice (True/False)	Enforces the field as required during server processing.
3. Read only	Choice (True/False)	Enforces the field as read-only during server processing.

4. Application	Global By Default	Defines the scope of the rule. It is by default chosen as application in which you are currently working

5. Execution Order and Conflict Management

Order of Execution (Field-Specific)

When multiple policies target the same field, the platform resolves conflicts based on the following precedence:

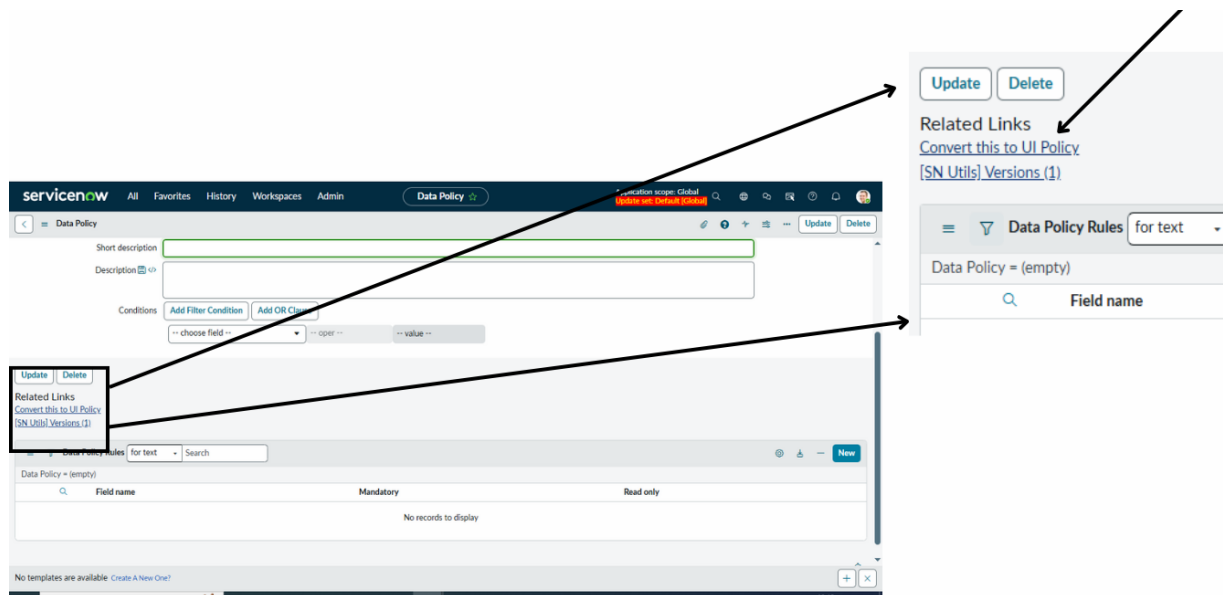
1. **Read-only:** A Read-only=True action from *any* active UI Policy takes precedence over Read-only=False and will lock the field.
2. **Visible:** A Visible=False action takes precedence and will hide the field. If one policy hides it, another cannot make it visible.
3. **Mandatory:** A Mandatory=True action takes precedence over Mandatory=False.
4. **Order Field:** After precedence rules, the explicit **Order** number is used as a tie-breaker, with lower numbers executing first. However, the *last* running policy's action for a given attribute (mandatory, read_only, visible) will *only* prevail if it is an enforcement (e.g., Mandatory=True) conflicting with a False/Leave Alone.

UI Policy vs. Client Script Execution

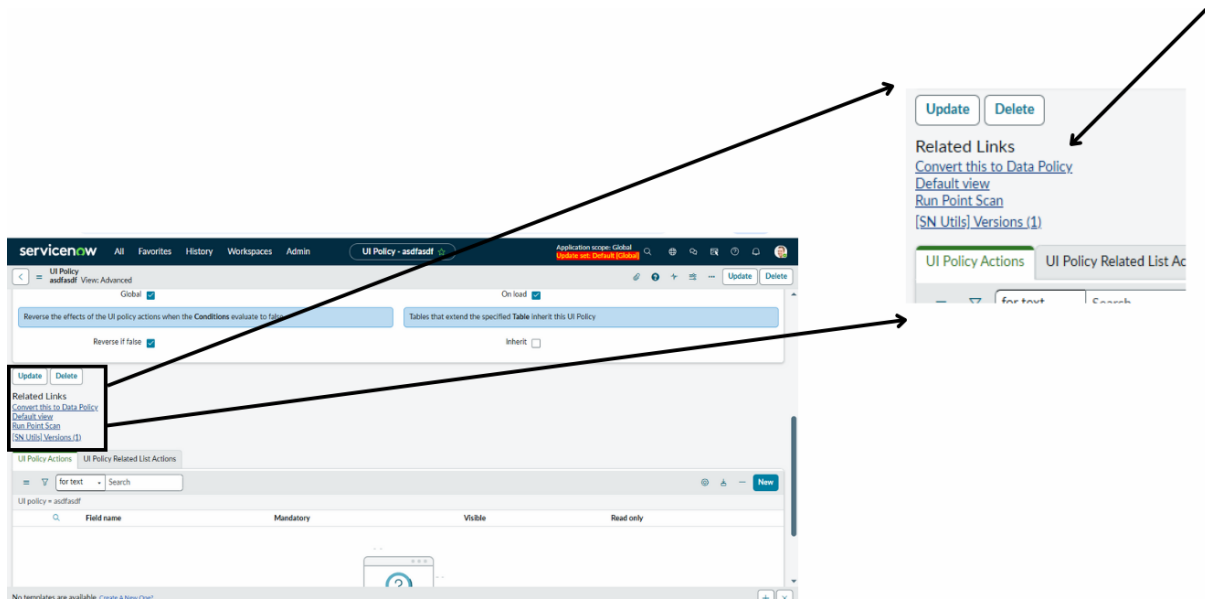
- **UI Policies** for the same field execute **after** any onLoad or onChange **Client Scripts** have run.
- **Best Practice:** Use UI Policies to enforce simple state changes and Client Scripts only for complex logic (AJAX, mathematical calculations, custom alerts) to maintain high performance and code separation.

6. Converting Data Policy → UI policy & UI policy → Data Policy

You can convert a data policy to ui policy but that will create a different record of ui policy in the sys_ui_policy table. But if you check the checkbox **Use as UI policy on client side**. That won't create a separate record but that data policy itself works as a ui policy if it has potential to do so. But a ui policy can't work as data policy. However, ui policy can be converted to data policy which will create a separate record in the data policy table. It can be done by using a related link which is shown below for both:



(Converting Data policy → UI policy)



(Converting UI policy → Data policy)

7. Advanced Topics and Lifecycle Management

7.1. Metadata and Migration

- **Metadata:** Both `ui_policy` and `data_policy` are records stored as metadata. They are fully captured in **Update Sets**.
- **Best Practice for Update Sets:** When migrating, ensure both the main policy record and its related actions/rules are committed. Verify the Order field in the target instance, as external conflicts can arise from policies not in the current update set.

7.2. Converting Data Policies to UI Policies (Internal Flow)

Step	Metadata Behavior	Result
1. Select Convert	A separate UI Policy record (<code>ui_policy</code>) with a new <code>sys_id</code> is	Enables separate client-side management (e.g.,

	created.	adding Visibility actions).
2. Mapping	Data Policy rules for Mandatory and Read-only are copied to the new UI Policy Actions.	The two policies are now independent records, but start with identical enforcement.
3. Server Rule	The original Data Policy remains active, ensuring server-side integrity is maintained independently.	The platform now enforces rules both client-side (UX) and server-side (Integrity).

7.3. Debugging (Non-Scripted)

1. **Enable Debugging:** Type **policy** in the Filter Navigator and click **UI Policy Debugging**.
2. **Execution Log:** When the form loads or a field changes, a debug window appears, logging every UI Policy that was evaluated (even inactive ones), the result of its Condition (True/False), and the resulting Actions applied to the fields.
3. **Troubleshooting:** If a field state is incorrect, check the debug log for **Conflicting Policies** (often identified by lower-order policies being overridden) or for a failed condition evaluation.